Tema 3

Introducción al lenguaje unificado del modelado (UML, UNIFIED MODELING LANGUAGE)

Entornos de Desarrollo

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 4
2. Características 5
3. Versiones 6
4. Diagramas UML 7
4.1 Diagramas estructurales 7
4.2 Diagramas de comportamiento 8
5. Utilización en metodologías de desarrollo orientado a objetos 10
6. Herramientas CASE con soporte UML 11
7. Resumen 14
Glosario 15
Bibliografía 16

Objetivos de la unidad

- Conocer el **Lenguaje Unificado de Modelado**, conocido por las siglas **UML**, desde una perspectiva conceptual, histórica y funcional, identificando los diferentes tipos de diagramas y tomando contacto con los más relevantes.
- Descubrir las herramientas CASE de asistencia a la ingeniería del *software* y tomar contacto con las mismas, tanto para la generación de código como para la realización de labores de ingeniería inversa.

Tech Hub <FP> euroformac

1. Introducción

El proceso de creación de una aplicación informática comienza en su definición en lenguaje natural y finaliza en la escritura del código fuente. A lo largo de este proceso debe existir una documentación que contenga de manera clara y precisa cada una de las características funcionales y técnicas del proyecto.

El **Lenguaje Unificado de Modelado (UML**, *Unified Modeling Language*) fue diseñado con el objetivo de proporcionar herramientas para la representación gráfica y textual de un sistema informático cuando este se realiza utilizando el paradigma de la orientación a objetos.

Tech Hub <FP> euroformac

2. Características

El **Lenguaje Unificado de Modelado** (**UML**) fue presentado por primera vez en 1997 por el *Object Management Group* (**OMG**), un consorcio sin ánimo de lucro creado para estandarizar tecnologías relacionadas con el paradigma de orientación a objetos.

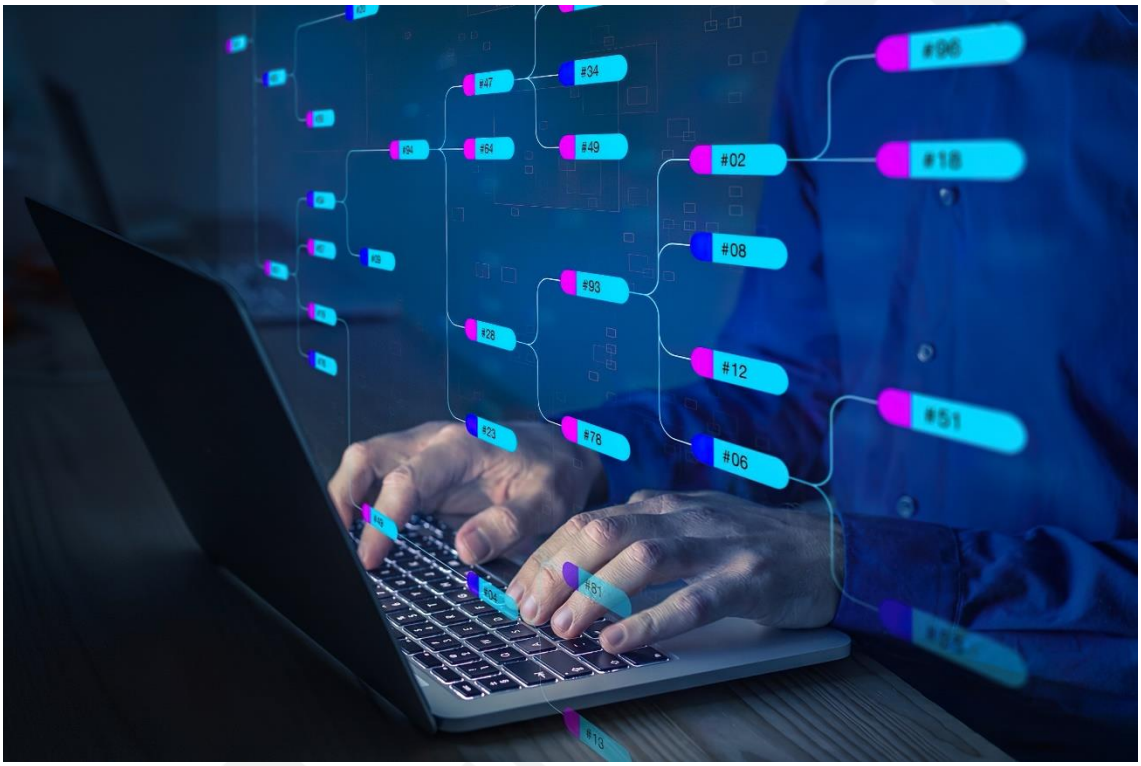
Logotipo de UML

A partir de 2004, la **Organización Internacional de Estandarización** (**ISO**, *International Organization for Standardization*), aprobó el lenguaje UML como estándar.

El lenguaje UML está formado por una relación de diagramas que permiten la documentación de diferentes aspectos de un sistema informático, tales como los procesos que contiene, los actores que participan en él, las interacciones entre actores y el sistema o los componentes *software* que lo componen.

3. Versiones

UML, desde su presentación en 1997 con la versión 1.1, ha ido evolucionando y mejorando. Desde diciembre de 2017 está vigente la versión 2.5.1. Hay que tener especial precaución a la hora de consultar documentación relacionada con UML, ya que entre versiones puede haber diferentes diagramas o tener enfoques distintos.



Más info

Se puede acceder al listado completo de versiones de UML en el siguiente enlace:

Ver enlace.

4. Diagramas UML

Debido a que un sistema *software* tiene muchos puntos de vista distintos, UML propone una serie de diagramas que tratan de cubrir todas las perspectivas del sistema.

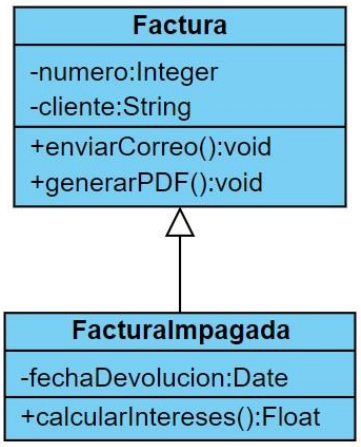
Los diagramas de UML se dividen en dos grandes grupos que, a su vez, engloban **15 diagramas**:

- **Diagramas estructurales**. Muestran el sistema desde un **punto de vista estático**: permiten representar los elementos que forman el sistema (clases, objetos, módulos o paquetes, por ejemplo) y cómo se organizan (relaciones de jerarquía e interacción).
- **Diagramas de comportamiento**. Muestran el sistema desde un **punto de vista dinámico**: permiten representar la actividad que realiza el sistema o, lo que es lo mismo, su funcionalidad. Contienen una subcategoría compuesta por los **diagramas de interacción**.

A continuación, se describen los diagramas contenidos por cada uno de estos grupos.

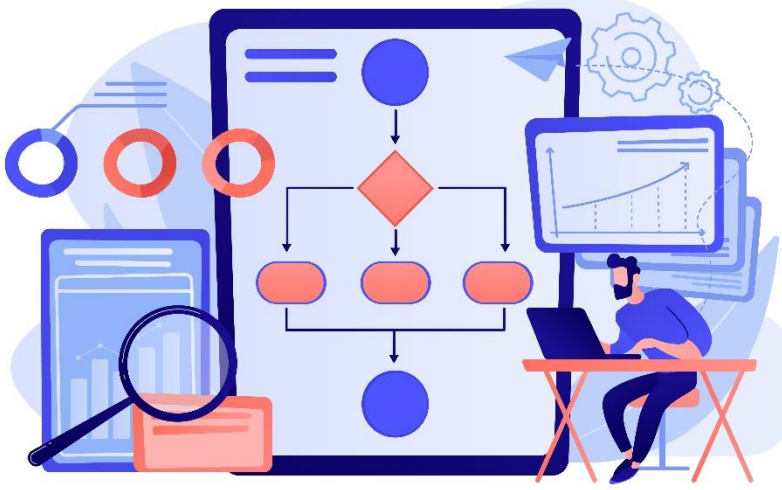
4.1 Diagramas estructurales

- **Diagrama de clases**. Permite representar las clases del sistema incluyendo atributos, métodos y las relaciones entre las mismas. Probablemente es el diagrama más habitual y útil. Muchas herramientas permiten generar código a partir de los diagramas de clase, mejorando la productividad del proceso de desarrollo.



Los diagramas de clases son los más utilizados de UML

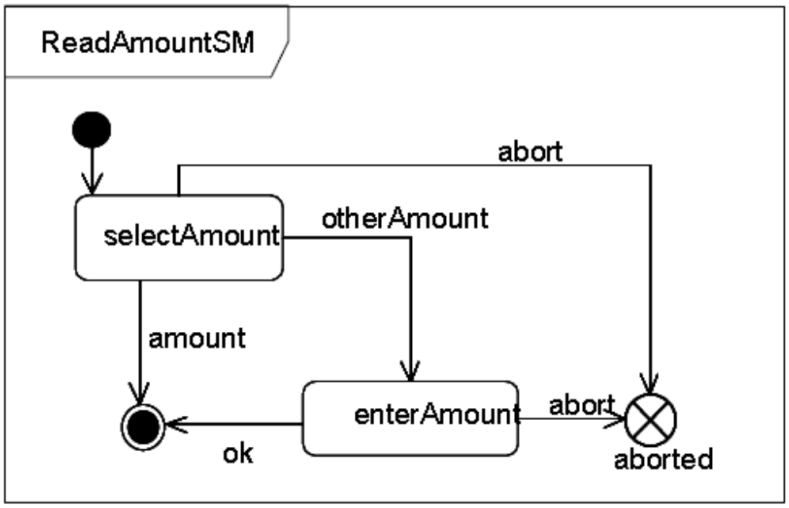
- **Diagrama de objetos**. Representa instancias de clases (objetos) en un instante concreto, incluyendo el valor de sus atributos.
- **Diagrama de componentes**. Muestra los componentes del sistema y sus relaciones (interfaces de comunicación o APIs). Los componentes pueden ser clases o agrupaciones lógicas de un nivel mayor como son los subsistemas.
- **Diagrama de lanzamiento o despliegue**. Este diagrama muestra, además de elementos *software*, el *hardware* del sistema. Tiene como vocación documentar la explotación o ejecución de una aplicación mostrando los elementos *software* (a alto nivel), los elementos físicos o *hardware* (servidores, bases de datos, etc.) y la relación entre ellos.
- **Diagrama de estructura compuesta**. Representa el interior de una clase, las interacciones internas y las conexiones de esta con el exterior.
- **Diagrama de paquetes**. Este tipo de diagramas permite representar los paquetes del sistema y las relaciones y dependencias que existen entre ellos.
- **Diagrama de perfiles**. Este diagrama proporciona una herramienta para la representación de particularidades aplicadas a partes del sistema, permitiendo proporcionar información referente a cómo dichas partes se particularizan para cada caso concreto (perfil).



4.2 Diagramas de comportamiento

- **Diagrama de actividad**. Representa de manera detallada (más que la proporcionada por los diagramas de caso de uso) la secuencia de pasos que se van a realizar para realizar una actividad.

- **Diagramas de interacción**.
  - **Diagrama de secuencia**. Permite documentar la secuencia de interacciones llevadas a cabo entre las clases de un sistema para la resolución de un problema.
  - **Diagrama de comunicación**. Representa las interacciones que realizan los objetos entre sí mediante el paso de mensajes.
  - **Diagrama general de interacción**. Muestra las interacciones que se desarrollan dentro del sistema desde una perspectiva de alto nivel.
  - **Diagrama de tiempos**. En este diagrama se representa el comportamiento de los componentes del sistema (normalmente objetos) durante un periodo de tiempo.
- **Diagrama de estados** o de máquina de estados. Permite representar los estados de un sistema, subsistema u objeto y las transiciones que se pueden producir entre estos.



Un diagrama de estados describe un ciclo de vida

- **Diagrama de casos de uso**. Representa cómo se relacionan los diferentes usuarios del sistema (conocidos como **actores**) con las diversas funciones de alto nivel que proporciona (caso de uso). Permite observar quién puede interactuar con el sistema y qué puede hacer con él.

5. Utilización en metodologías de desarrollo orientado a objetos

Una metodología de desarrollo orientado a objetos es un conjunto de métodos, modelos y reglas que permiten construir un sistema *software* de manera ordenada y correcta.

Existen muchas metodologías y pueden existir tantas variantes de estas como proyectos o personas las utilizan. Las metodologías establecen el marco de trabajo, pero este siempre puede necesitar ajustes para adaptarse a la realidad de un proyecto determinado.

Algunas metodologías de desarrollo orientado a objetos clásicas son **OOSE** (*Object Oriented Software Engineering*), **OMT** (*Object Modeling Technique*) o **Booch**.

UML no sigue ninguna metodología en concreto, sino que proporciona herramientas para aplicar en estas. Gracias a UML se pueden representar las diferentes partes estáticas o dinámicas del sistema y encajar dichas representaciones como parte del trabajo realizado al seguir una metodología. Por ejemplo, la representación de las clases de un sistema se puede realizar utilizando el **diagrama de clases** de UML siempre que este sea compatible con las necesidades de la metodología.

6. Herramientas CASE con soporte UML

UML es una especificación e indica qué debe representar cada diagrama, con qué símbolos y en qué casos, pero no proporciona los medios para realizar el trabajo.

La generación de los diagramas de UML podría realizarse de manera manual, con papel y bolígrafo, pero esto sería poco eficiente cuando se puede disponer de herramientas que ayuden a llevar a cabo esta tarea.

Estas herramientas pueden ser desde editores como **PowerPoint,** del paquete Microsoft Office, o **Draw** del paquete LibreOffice, sin soporte específico o completo para UML, hasta las conocidas como herramientas **CASE** (*Computer Aided Software Engineering*) diseñada y construidas específicamente para ayudar en diferentes fases del proceso de creación de *software* y que pueden proporcionar soporte específico para UML.

Además de otras funciones relacionadas con la construcción del *software*, las herramientas CASE pueden dar soporte a la creación de uno o más tipos de diagrama de UML, existiendo incluso componentes que se pueden añadir a algunos **IDE** (*Integrated Development Environment*) convirtiéndolos, aunque sea parcialmente, en herramientas CASE.

Sabías que

La definición de herramienta CASE es lo suficientemente ambigua como para que se pueda considerar como tal a un editor de código fuente con capacidades relacionadas con las tareas de la ingeniería del software. Es por esta razón que, en ocasiones, editores o herramientas no específicamente creadas para asistir en las labores del diseño y desarrollo del software se consideren herramientas CASE al incluir funciones relacionadas con estas actividades.

Algunas herramientas con soporte para UML son, por ejemplo, ***Software* Ideas Modeler** (dispone de versión gratuita para uso no profesional), **Smartdraw, Visual Paradigm**, **Microsoft Visio, Lucidchar**, **Gleek**, **Gliffy** o **diagrams.net**.

Más info

Se puede obtener más información referente a qué es una herramienta CASE en el siguiente artículo:

Ver enlace.

Para ilustrar, aunque sea ligeramente, la utilidad de las herramientas CASE, se ha instalado ***Software* Ideas Modeler** y creado un diagrama de clases. En dicho diagrama se ha diseñado una clase con el nombre -Electrodoméstico- que contiene los atributos -marca- y -modelo-y el método -mostrarModelo-.

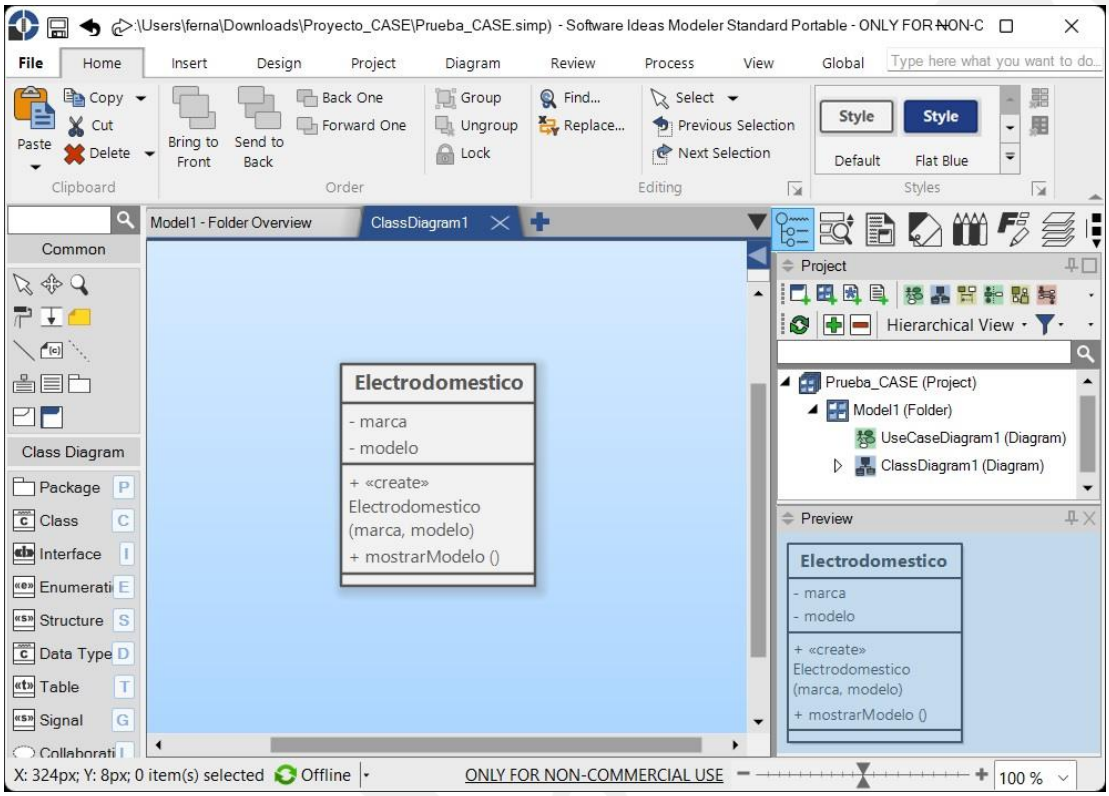


Diagrama de clases creado con Software Ideas Modeler

Una vez diseñada la clase, utilizando el generador de código incluido en la herramienta, se ha generado el código fuente para el lenguaje Python (se puede generar para muchos más lenguajes, como C#, C++, Java o PHP, por citar algunos ejemplos). El resultado se muestra en la siguiente captura de pantalla.

Electrodomestico.py ×

C: > Electrodomestico.py >

```python
"""

-- Generated by Software Ideas Modeler (Trial Version)
"""
class Electrodomestico:
    def __init__(self, marca, modelo):

        self.__marca =marca

        self.__modelo =modelo

    def mostrarModelo (self):
        # implementation
        pass

```

Código fuente Python generador por Software Ideas Modeler

Las herramientas CASE también suelen disponer de herramientas para realizar **ingeniería inversa**, generando los diagramas de clase a partir del código fuente ya programado. Esto facilita la documentación de aplicaciones que no estaban previamente documentadas.

7. Resumen

La complejidad de las aplicaciones informáticas puede ser muy importante, por lo que es necesario aplicar técnicas de ingeniería de software para tener control sobre los proyectos.

UML es un lenguaje de modelado estandarizado y validado a lo largo de sus muchos años de existencia por gran cantidad de empresas y proyectos. Gracias a UML y a sus diferentes tipos de diagramas, se puede plasmar en documentos prácticamente todos los aspectos de un proyecto de software, facilitando así su construcción, explotación y mantenimiento.

Las conocidas como herramientas CASE, facilitan las tareas relacionadas con la gestión de proyectos de software, en particular los relacionados con los diagramas UML.

Tech Hub <FP> euroformac

Glosario

- **Atributo**. En programación orientada a objetos, cada una de las características de una clase.
- **Clase**. En programación orientada a objetos, la representación de un elemento del sistema.
- **Despliegue**. En desarrollo de *software*, la actividad de instalar, configurar y poner en marcha una aplicación.
- **Herencia**. En programación orientada a objetos, es una relación en la que una clase (derivada o hija) adquiere atributos y métodos de otra (clase base o superclase) y amplía su funcionamiento.
- **Herramienta CASE (*Computer Aided Software Engineering*)**. Es una aplicación informática que proporciona mecanismos para ayudar a diseñar y desarrollar sistemas *software*.
- **Ingeniería inversa**. Actividad que consistente en, a partir de un sistema ya construido, obtener información sobre su origen. En ingeniería de *software* se suele referir a la obtención de información de diseño (diagramas de clase, por ejemplo) partiendo del código fuente.
- **Método**. En programación orientada a objetos, la parte de la clase que implementa su comportamiento (las acciones que puede realizar.
- **Objeto**. Es la concreción de una clase. También se conoce como instancia.

Bibliografía

https://themeforest.net/user/dan_fisher. (n.d.). Welcome To UML Web Site! https://www.uml.org/

https://themeforest.net/user/dan_fisher. (n.d.-a). OMG | Object Management Group. https://www.omg.org/

About the Unified Modeling Language Specification Version 2.5.1. (n.d.). https://www.omg.org/spec/UML/

IBM Developer. (n.d.). https://developer.ibm.com/articles/the-class-diagram/